[ ]:
```
-- EJERCICIO 1
```

[12]:
```
a = [1,2,3]
b = [4,5]
c = a ++ b
return c
```

```
[1,2,3,4,5]
```

[13]:
```
:t True
:t "Hola mundo"
:t [1,2,3]
```

```
Verdadero :: Booleano

"Hola mundo" :: String

[1,2,3] :: para todo {a}. Num a => [a]
```

*[14]:
```
-- 2. ¿Qué significa que Haskell sea un lenguaje fuertemente tipado y evaluado perezosamente?
-- R// Significa que el lenguaje es extremadamente estricto con las reglas de sus datos, pero muy flexible y eficiente sobre cuándo procesa esos datos
```

[24]:
```
doble :: Int -> Int
doble x = 2 * x
doble 2
```

```
4
```

[25]:
```
esMayorEdad :: Int -> Bool
esMayorEdad edad = edad >= 18
esMayorEdad 15
```

```
FALSO
```

[34]:
```
areaCirculo :: Float -> Float
areaCirculo r =  (r * r ) * pi
areaCirculo 5
```

```
78.53982
```

[35]:
```
--  ¿Qué diferencias encuentras entre los tipos Int y Float?
-- R// Que int es para valores especificos enteros y float son mas para valores inexactos y ambos tienen sus ventajas y desventajas dependiendo del problema.
```

[36]:
```
-- EJERCICIO 2
```

[37]:
```
head [10,20,30,40]
tail [10,20,30,40]
take 3 [1..10]
drop 4 [1..10]
length [1,2,3,4,5]
```

```
10
[20,30,40]
[1,2,3]
[5,6,7,8,9,10]
5
```

[38]:
```
-- ¿Qué hace la expresión [1..10] y cómo se relaciona con la evaluación perezosa?
-- R// Que solo toma los valores de un numero n que le damos nosotros de una lista, es como definir un rango y se relaciona en que es flexible y eficiente.
```

[50]:
```
primerElemento :: [a] -> a
primerElemento (x:_) = x
primerElemento [85,2,3,4]
```

```
85
```

[57]:
```
productoLista :: [Int] -> Int
productoLista [] = 1
productoLista (x:xs) = x * productoLista xs
productoLista [2,3,4]
```

Utilice foldr

Encontró: **Por qué no:**

lista de productos [] = 1 lista de productos (x : xs) = x * lista de productos xsproductoLista xs = foldr (*) 1 xs

```
24
```

[61]:
```
eliminarNegativos :: [Int] -> [Int]
eliminarNegativos xs = filter (>= 0) xs
eliminarNegativos [1,-2,3]
```

Eta reduce

Found: **Why Not:**

eliminarNegativos xs = filter (>= 0) xseliminarNegativos = filter (>= 0)

```
[1,3]
```

[67]:
```
invertir :: [a] -> [a]
invertir [] = []
invertir (x:xs) = invertir xs ++ [x]
invertir [1,2,3,4]
```

```
[4,3,2,1]
```

[68]:
```
-- EJERCICIO 3
```

[69]:
```
cuadrado :: Int -> Int
cuadrado n = n * n

suma :: Int -> Int -> Int
suma a b = a + b

sumaCuadrados :: Int -> Int -> Int
sumaCuadrados x y = suma (cuadrado x) (cuadrado y)
sumaCuadrados 2 5
```

```
29
```

[71]:
```
duplicar :: Int -> Int
duplicar n = n * 2

-- Forma simple
dobleListaSimple :: [Int] -> [Int]
dobleListaSimple = map duplicar

--Forma compleja
dobleListaCompleja :: [Int] -> [Int]
dobleListaCompleja xs = map duplicar xs
dobleListaSimple [1,2,3]
```

Eta reduce

Found: **Why Not:**

```
dobleListaCompleja xs = map duplicar xsdobleListaCompleja = map duplicar
```

```
[2,4,6]
```

[1]:
```
take 5 (map (*2) [1..])
```

```
[2,4,6,8,10]
```

[3]:
```
misterio x y = if x > 10 then y * 2 else x + y
```

[4]:
```
misterio (5+7) (4*3)
```

```
24
```

1.

A.

```
[2]: doble = \x -> 2 * x
     doble 5
```

Redundant lambda

Found: Why Not:

```
doble = \ x -> 2 * xdoble x = 2 * x
```

Avoid lambda

Found: Why Not:

```
\ x -> 2 * x(2 *)
```

```
10
```

B.

```
[3]: cubo = \x -> x ^ 3
     cubo 3
```

Redundant lambda

Found: Why Not:

```
cubo = \ x  -> x ^ 3cubo x = x ^ 3
```

Avoid lambda using `infix`

Found: Why Not:

```
\ x -> x ^ 3(^ 3)
```

```
27
```

c.

```
[4]: mitad = \x -> x / 2
     mitad 8
```

Redundant lambda

Found: Why Not:

```
mitad = \ x  -> x / 2mitad x = x / 2
```

Avoid lambda using `infix`

Found: Why Not:

```
\ x -> x / 2(/ 2)
```

```
4.0
```

D.

```
[5]: esPositivo = \x -> x > 0
     esPositivo (-2)
```

Redundant lambda

Found: Why Not:

```
esPositivo = \ x  -> x > 0esPositivo x = x > 0
```

Avoid lambda using `infix`

Found: Why Not:

```
\ x -> x > 0(> 0)
```

```
False
```

E.

```
[6]: enIntervalo = \x -> x >= 10 && x <= 20
     enIntervalo 15
```

Redundant lambda

Found: Why Not:

```
enIntervalo = \ x -> x >= 10 && x <= 20enIntervalo x = x >= 10 && x <= 20
```

```
True
```

2.

A.

```
suma = \x y -> x + y
suma 4 7
```

Redundant lambda

Found: Why Not:

```
suma = \ x y -> x + ysuma x y = x + y
```

Avoid lambda

Found: Why Not:

```
\ x y -> x + y(+)
```

11

B.

```
multiplicar = \x y -> x * y
multiplicar 3 6
```

Redundant lambda

Found: Why Not:

```
multiplicar = \ x y -> x * ymultiplicar x y = x * y
```

Avoid lambda

Found: Why Not:

```
\ x y -> x * y(*)
```

18

C.

```
potencia = \x y -> x ^ y
potencia 2 5
```

Redundant lambda

Found: Why Not:

```
potencia = \ x y -> x ^ ypotencia x y = x ^ y
```

Avoid lambda

Found: Why Not:

```
\ x y -> x ^ y(^)
```

32

D.

```
maximo = \x y -> if x > y then x else y
maximo 9 12
```

Redundant lambda

Found: Why Not:

```
maximo = \ x y -> if x > y then x else ymaximo x y = if x > y then x else y
```

Use max

Found: Why Not:

```
if x > y then x else ymax x y
```

12

E.

```
sonIguales  = \x y -> x == y
sonIguales 7 7
```

Redundant lambda

Found: Why Not:

```
sonIguales = \ x y -> x == ysonIguales x y = x == y
```

Avoid lambda

Found: Why Not:

```
\ x y -> x == y(==)
```

True